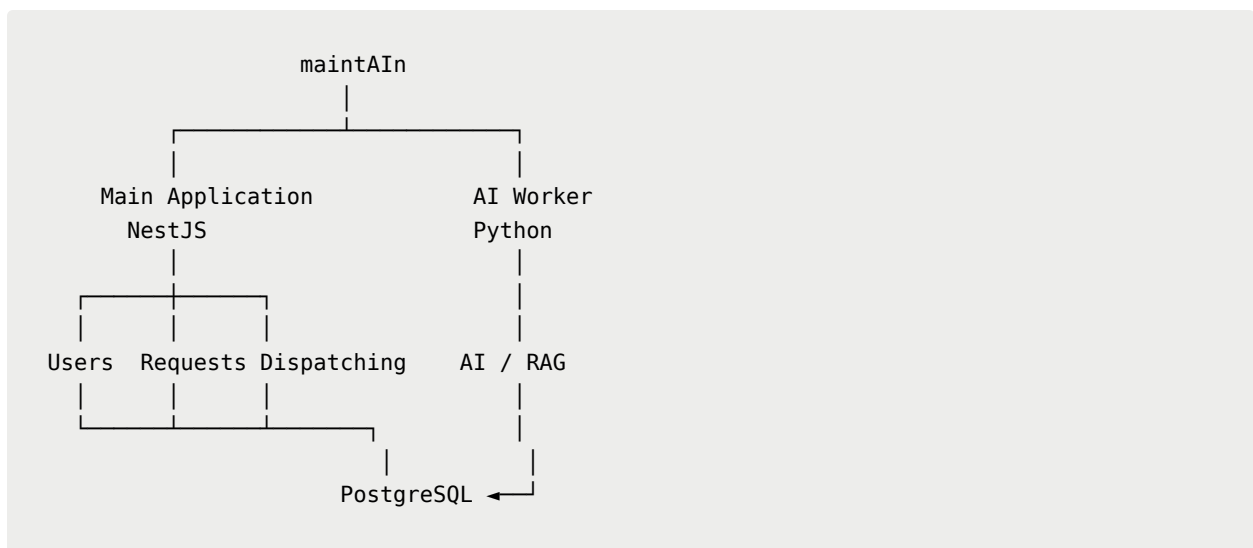


System Design decision:

For **maintAI**, I think A — **Modular Monolith + Separate AI Worker** is the most appropriate architecture.

The important distinction is that we're **not building a complicated microservices system**.

Final architecture decision #1



Why it fits your project

NestJS stays responsible for the actual system:

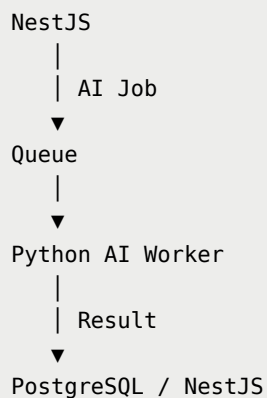
- Authentication & authorization
- Customers
- Technicians
- Maintenance requests
- Dispatching
- Technician ranking
- Reviews
- Scheduling

- Manager analytics
- Business rules
- Database
- API

Python AI Worker handles the AI-heavy work:

- Image + text analysis
- Follow-up question generation
- Maintenance classification
- Possible-cause analysis
- RAG retrieval/processing
- AI response generation
- Potential future ML models

And communication can be:



This gives you a clean separation:

NestJS decides what the system is allowed to do.

AI suggests what might be useful.

That distinction is especially important for maintAI because technician assignment, permissions, safety rules, request states, etc. should **not be**

controlled directly by an LLM.

One important constraint

Don't turn the Python worker into a giant independent application.

For the first version, it should basically be:

AI processing service → receives structured job → processes → returns structured result.

That keeps the project achievable within your timeframe.

So I'd lock:

Decision #1: Modular Monolith + Separate AI Worker

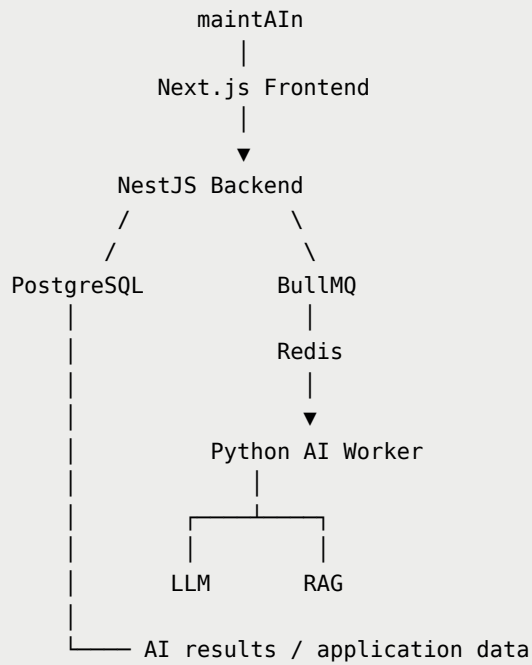
So far:	
Decision	Choice
Backend architecture	Modular Monolith + Separate AI Worker
Backend framework	NestJS + TypeScript
ORM	Prisma
Database	PostgreSQL

Decision #3 — AI Job Queue

Redis + BullMQ

We'll use it specifically for **background AI jobs**.

Your architecture now:



The rule we'll follow

Synchronous → normal backend operations

Login
Get requests
Update profile
View technician
Assign technician
Update status
Submit review

Asynchronous → AI/heavy processing

Analyze customer request
Analyze image
Generate follow-up questions
Run RAG analysis
Generate maintenance report
Generate technician job brief

This keeps the architecture clean rather than putting everything into the queue.

Current decisions

#	Decision	Choice
1	Backend architecture	Modular Monolith + Separate AI Worker
2	ORM	Prisma
3	AI job processing	Redis + BullMQ

Decision #4 — RAG / Vector Storage

Choice: PostgreSQL + pgvector

We will **not** use a separate vector database.

Your PostgreSQL database will contain both:

```
Application Data
├─ Users
├─ Customers
├─ Technicians
├─ Requests
├─ Equipment
├─ Reviews
└─ ...

AI Knowledge
├─ Documents
├─ Knowledge Chunks
├─ Embeddings
└─ Verified Maintenance Cases
```

The RAG pipeline will be:

```
Maintenance Documents
↓
Extract text
↓
Chunking
↓
Embeddings
↓
```

PostgreSQL + pgvector



RAG Retrieval



Maintenance Agent

Your exact RAG scope

We're **not** building a general maintenance AI.

For the MVP:

1. AC / HVAC

- Not cooling
- Weak airflow
- Ice formation
- Water leakage
- Strange noises
- Compressor-related issues
- Refrigerant-related issues
- Common electrical/control issues
- Basic maintenance

2. Refrigerators

- Not cooling
- Excessive ice/frost
- Water leakage
- Strange noises
- Compressor-related issues
- Thermostat/control issues
- Refrigerant-related issues
- Basic maintenance

3. Electrical

Only **common/famous electrical maintenance problems relevant to your platform**, not a complete electrical engineering knowledge base.

Examples:

- Power failure
- Circuit breaker trips
- Fuse issues
- Outlet/switch problems
- Basic appliance electrical faults
- Common wiring/safety situations

4. Plumbing

Not included.

So the system boundary is:

AC/HVAC + Refrigeration + selected common electrical maintenance.

How we give it the data

You should use **two sources of knowledge**.

A. Technical documentation

Collect a small number of reliable documents:

AC/HVAC

- └─ Manufacturer manuals
- └─ Troubleshooting guides
- └─ Maintenance procedures
- └─ Safety documentation

Refrigeration

- └─ Manufacturer manuals
- └─ Troubleshooting guides
- └─ Maintenance procedures
- └─ Safety documentation

```
Electrical
├─ Basic troubleshooting references
├─ Safety documentation
└─ Relevant appliance/electrical guides
```

Don't make hundreds of documents.

For your MVP, I'd rather have **a carefully selected collection of high-quality documents**.

B. Verified maintenance cases

These are particularly important for your feedback mechanism.

Example:

```
Case #001

Equipment:
AC

Customer symptoms:
- AC running
- Not cooling
- Ice on pipe

AI analysis:
- Restricted airflow
- Dirty filter
- Refrigerant-related issue

Technician:
Actual fault → Dirty filter

AI analysis useful:
YES / NO

Resolution:
Filter cleaned/replaced

Technician feedback:
...
```

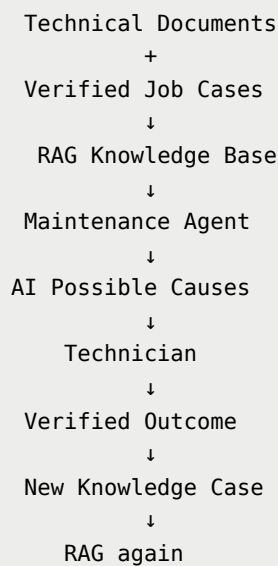
After verification, this becomes a **verified case** that can potentially be retrieved by RAG for future similar cases.

The important distinction

Your documents are **initial knowledge**.

Your technician cases become **your system's domain-specific knowledge**.

So eventually:



That gives maintAI a real **feedback-driven improvement loop** without requiring you to train an LLM.

And one thing we should enforce

The AI should **never treat its own previous answer as truth**.

Only appropriately verified technician/manager outcomes should be promoted into the trusted case knowledge.

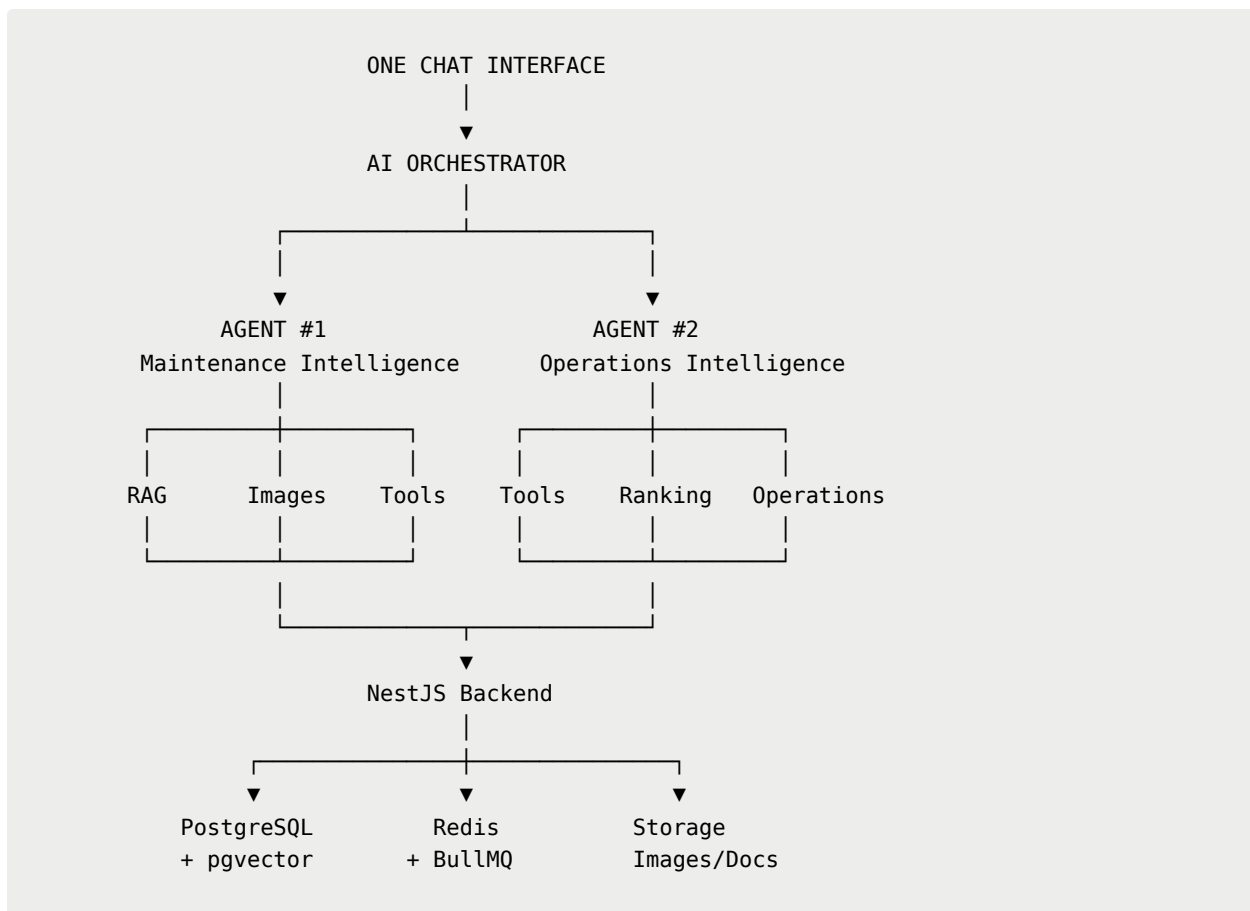
Current decisions

#	Decision	Final choice
1	Backend architecture	NestJS Modular Monolith + Python AI Worker
2	ORM	Prisma
3	AI job processing	Redis + BullMQ

#	Decision	Final choice
4	RAG/vector storage	PostgreSQL + pgvector
5	RAG scope	AC/HVAC + Refrigerators + selected common Electrical
Excluded	Plumbing	✗

maintAI n AI Architecture — Decision #7

You will have **one unified chat interface**, with a central **AI Orchestrator** routing requests to **two specialized agents**.



Agent #1 — Maintenance Intelligence

Primarily for the **customer side**.

It handles:

- Customer conversation
- Text analysis
- Image analysis
- Identifying the maintenance category
- Urgency/status assessment
- Safety concerns
- Asking necessary follow-up questions
- Possible causes
- RAG-based technical evidence
- Preparing the complete maintenance case
- Recommending the appropriate maintenance team/category

It should **not claim a confirmed diagnosis**.

Agent #2 — Operations Intelligence

Primarily for the **dispatcher/management side**.

Once Agent #1 produces the case:

```
Customer
  ↓
Agent #1
  ↓
Complete Maintenance Case
  ↓
Dispatcher
  ↓
Agent #2
```

Agent #2 helps with:

- Technician/team matching
- Skills

- Availability
- Current workload
- Previous similar jobs
- Efficiency/performance
- Required tools
- Tool availability
- Estimated cost
- Scheduling considerations
- Explaining why a technician/team is suitable

The actual ranking/calculation will eventually be handled by **NestJS backend logic**, with Agent #2 explaining and interacting with it.

The important decisions we've now established

#	Decision	Choice
1	Backend	NestJS Modular Monolith + Python AI Worker
2	ORM	Prisma
3	AI jobs	Redis + BullMQ
4	RAG storage	PostgreSQL + pgvector
5	Knowledge scope	AC/HVAC + Refrigerators + selected Electrical
6	Model strategy	Primary LLM + fallback LLM + local embedding model
7	AI architecture	One Orchestrator + 2 specialized agents
8	AI ↔ backend tools	Later

One clarification about #6

We have **not yet selected the actual LLM provider/model**. We only decided the architecture should support:

Primary LLM
↓
Fallback LLM

And a **local pretrained embedding model** for RAG.

So we haven't locked something like Claude, Gemini, or OpenAI yet.

This gives us a solid foundation. The next decisions should now become **more concrete and implementation-level**, rather than changing the overall architecture.

Decision 8 — Agent #2 Technician Recommendation

Agent #2 uses backend tools to get the information it needs:

- technician skills
- similar experience
- availability
- workload/distance
- required tools

It does **not** directly access or manipulate the database.

Decision 9 — Technician Ranking

Only 4 weighted factors:

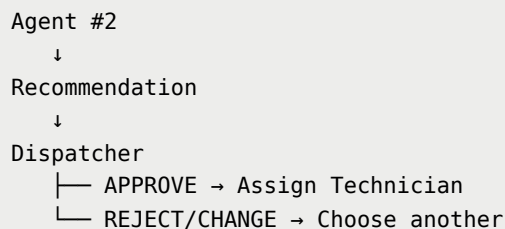
Normal: 35% Skill · 25% Experience · 25% Availability · 15% Workload/Distance

Emergency: 30% Skill · 20% Experience · 35% Availability · 15%
Workload/Distance

These are the two you meant.

Decision 10 — Dispatcher Approval & Assignment

- Agent #2 **recommends** a technician.
- Dispatcher has full access to the recommendation and case details.
- Dispatcher can **Approve** → technician gets assigned.
- Dispatcher can **Reject/Change** → dispatcher selects another technician.
- **No automatic assignment.**
- AI never bypasses the dispatcher.



This also keeps the human decision point clear in maintain.

Decision 11 — Technician Job Alert + Limited AI Assistance

The technician's **main interface is not an AI chatbot**. The technician receives a clear **Job Alert** from the dispatcher.

Job Alert contains exactly what he needs

- **Job / request ID**
- **Customer location**
- **Equipment** — e.g. AC, refrigerator
- **Problem summary**
- **Symptoms**
- **Images provided by customer**

- **Safety warnings**
- **Scheduled time / urgency**
- **Relevant customer/equipment history**
- **Possible causes identified by maintAI**
- **Suggested tools / parts**, when available
- **Dispatcher notes**

The technician can then **Accept / Start / Complete** the job.

Limited AI for the technician

This is the important distinction you made:

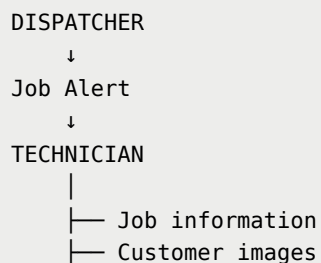
The AI is **supportive, not central**.

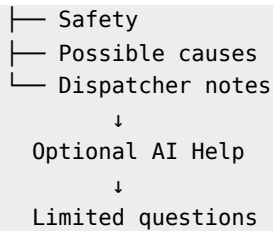
The technician can upload/provide the **dispatcher's PDF/schema/job information** to the AI assistant, and ask limited questions such as:

- | "Based on this case, what should I check first?"
- | "What tool would normally be needed for this check?"
- | "Explain this possible cause."
- | "What safety precaution applies here?"
- | "Show me the relevant troubleshooting information."

The AI works **from the information already provided + approved maintAI knowledge/RAG**.

It does **not** independently manage the job or make decisions for the technician.





Why this is better for maintAI

It prevents the project from becoming:

| "AI assistant for everyone."

Instead:

Customer → AI is important

Dispatcher → AI is important

Technician → AI is optional/supportive

And the technician's **actual diagnosis and resolution remain authoritative**. That is also what gives you the verified outcome for your later knowledge/feedback loop.

Decision 11 = LOCKED.

Decision 13 — Manager AI: Locked

We **do not make analytics a major component** of maintAI.

The Manager AI focuses mainly on:

- Operational questions
- Requests/cases
- Technician information
- Repeated problems
- Basic statistics when useful
- Controlled database queries

Pricing

We don't build a complex pricing/financial system.

If a calculation is needed, we can store the required values in PostgreSQL, for example:

```
Service type
Base price
Parts cost
Estimated labor cost
Technician rate
```

Then the backend can perform simple calculations such as:

```
Total estimated cost =
Base service + Parts + Labor
```

The AI can **explain/display the result**, but the backend/database provides the actual numbers.

So we avoid:

- Complex financial analytics
- Dynamic pricing algorithms
- Revenue forecasting
- Advanced business intelligence
- Complicated dashboards

This keeps the project within the **2–2.5 month scope** while still allowing price information wherever the workflow genuinely needs it.

Decision 13 locked.

Decision 14 — RAG Knowledge Base

- Focus on **AC/HVAC, refrigerators, and selected electrical problems.**

- Gather **reliable sources**: manufacturer manuals, technical documents, safety documentation, and later verified technician cases.
- Extract only the **useful maintenance information**.
- Convert it into our own structure:
 - Problem
 - Symptoms
 - Possible causes
 - Evidence
 - Safe checks
 - Tools/measurements
 - Safety warnings
 - Escalation conditions
- Do **not invent missing information**.
- Embed the structured knowledge into **PostgreSQL + pgvector**.
- Later, add **verified technician cases** as trusted maintAI knowledge.

Decision 14 = LOCKED.

great so now give me 15

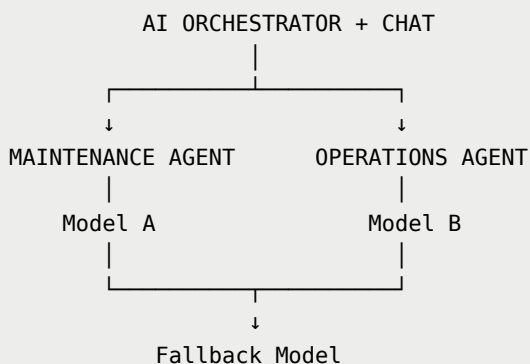
Decision 15 — AI Architecture & Model Strategy

Locked:

- **One central AI Orchestrator + Chat** for all users.
- **Only 2 specialized agents**:
 - **Agent #1 — Maintenance Intelligence**
 - **Agent #2 — Operations Intelligence**
- The Orchestrator decides **when an agent is needed** and routes the conversation accordingly.

- Each agent can use a **model suited to its specific work**.
- **One common fallback model** is available if a primary model fails.
- **One shared RAG system**, mainly for the Maintenance Agent.
- RAG uses **PostgreSQL + pgvector**.
- Operations mainly uses **controlled backend tools/live database data**, not a separate RAG.
- Manager does **not** have a third AI agent. Manager insights are accessed through the Orchestrator using controlled backend/analytics capabilities.
- The system uses:
 - System prompts
 - RAG
 - Backend tools
 - Structured outputs
 - Permissions
 - Safety/business rules
- Exact AI models/providers will be selected later based on their capabilities and project requirements.
- **Fine-tuning is not required for the MVP.**

Core architecture



Decision 16 — LOCKED

AI ↔ Backend Tools

- AI communicates with the backend **only through controlled tools**.
- AI has **no direct PostgreSQL/database access**.
- Backend validates permissions and inputs.
- Agent #1 uses maintenance/RAG-related tools.
- Agent #2 uses operational/technician tools.
- Manager uses controlled insight tools through the Orchestrator.
- AI cannot independently approve, assign, or bypass business rules.

Customer — AI can access:

- Their maintenance requests
- Status of their requests
- Their equipment information/history
- Possible problems identified by the Maintenance Agent

Customer AI does NOT provide:

- Step-by-step repair guidance
- Instructions to personally fix the equipment

The AI can explain what it identified, but the actual maintenance is handled through the maintenance workflow.

Technician — AI can access:

- Their assigned jobs
- Job information

- Maintenance case information provided to them
- Relevant equipment/history
- Limited AI assistance based on the job information and approved knowledge

Dispatcher — AI can access:

- Maintenance cases
- Relevant customer/request information
- Equipment/history needed for dispatching
- Technician information
- Availability
- Operations/assignment information

Manager — AI can access:

- Company-level operational information
- Maintenance statistics/insights
- Technician performance information
- Request/problem trends
- Other authorized managerial data

Core rule

```
AI access = user's role + authorized data
```

And again, this is **only the AI permission layer**. The actual application-level permissions will be designed separately in the system/security design.

Decision 17 = LOCKED.

Decision 18 — Maintenance Request Lifecycle

There are **two ways to create a maintenance request**.

Path A — Manual

The customer already knows the problem and wants to handle it directly.

```
Customer
  ↓
Choose Team
  ↓
Choose / Request Technician
  ↓
Fill Maintenance Form
  ↓
Submit Request
  ↓
Dispatcher Reviews
  ↓
Approve
  ↓
Technician Receives Job Alert
  ↓
Technician Accepts
  ↓
Job
```

No AI is required.

Path B — AI-Assisted

The customer isn't sure about the problem or wants help preparing the request.

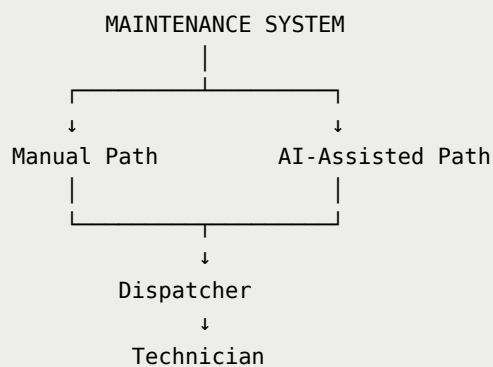
```
Customer
  ↓
AI Chat
  ↓
Text + Image + Questions
  ↓
AI analyzes the problem
  ↓
AI prepares the Maintenance Form/Case
  ↓
Customer Reviews
  ↓
Send / Cancel / Edit
  ↓
Dispatcher Reviews
  ↓
Approve
```

↓
Technician Receives Job Alert
↓
Technician Accepts

The customer **always remains in control** of whether to send the AI-generated request.

Important principle

AI can assist at different points, but **the core system works without AI**.



This is actually a strong architectural choice because if the AI is unavailable, **maintAI still functions as a normal maintenance platform**.

And the same principle applies to the other roles: **AI is an optional intelligence layer, not a dependency for basic system operations**.

Decision 18 = LOCKED

Decision 19:

Core — absolutely needed

Around **20–25 tables**:

users
roles
user_roles

```
customer_profiles
customer_addresses

technician_profiles
dispatcher_profiles

teams
technician_teams
skills
technician_skills
technician_availability

equipment
equipment_types

maintenance_requests
maintenance_cases
case_status_history
attachments

assignments
job_reports

reviews
notifications
audit_logs
```

AI/RAG — needed for your AI architecture

```
ai_conversations
ai_messages
ai_runs
ai_tool_calls
ai_feedback

knowledge_sources
knowledge_documents
knowledge_chunks
knowledge_embeddings
```

Optional / activate when needed

```
maintenance_symptoms
maintenance_causes
case_evidence
```

```
job_parts
service_prices
job_costs
```

Decision 20 — Security: strengthened

We keep the authentication/RBAC architecture we accepted, but security becomes a **visible system capability**.

Focus on practical security features that fit your project:

- **RBAC** — strict role permissions
- **AI permission enforcement** — AI inherits the user's permissions
- **Protected AI tools** — agents cannot call unauthorized backend functions
- **Secure file/image access** — users only access permitted attachments
- **Input validation** — backend validates requests and AI tool inputs
- **API protection** — authentication, authorization, rate limiting where appropriate
- **Password security** — strong hashing and secure credential handling
- **Secrets management** — API keys/database credentials never in Git/frontend
- **Audit logs** — important actions recorded
- **AI action traceability** — record important AI tool calls/recommendations
- **Database protection** — Prisma/backend layer, no direct client database access
- **AI safety boundaries** — especially around maintenance/safety advice
- **Secure error handling** — don't expose database/internal errors to users

Security can become part of the demo

For example, you can demonstrate:

| **Technician tries to access another technician's job → denied.**

| **Customer tries to access another customer's maintenance case → denied.**

| **AI attempts an unauthorized backend tool → backend rejects it.**

| **Dispatcher approves an assignment → action appears in audit history.**

That makes security **demonstrable functionality**, rather than a slide saying "our system is secure."

We can also later create a dedicated **Security Architecture** section in the final blueprint.

Decision 20 = LOCKED with security as a significant feature.

Decision 21 — File & Image Storage

- **Customer images:** stored and attached to the maintenance request.
- **Technician/job images:** can be stored as part of the completed job.
- Actual files → **object storage**.
- File metadata/reference → **PostgreSQL**.
- Access controlled through the **NestJS backend + RBAC**.
- PDFs/documents attached to normal cases are **optional/later**.
- RAG source documents are handled as part of the **RAG knowledge pipeline**, not ordinary case attachments.

And the complete case itself remains represented through the database relationships we discussed:

Request → AI analysis → Dispatcher → Assignment → Technician → Job report → Completion → Feedback/history.

21 = LOCKED.

Exactly. **Decision 22 = A.**

We keep the system simple:

- **Simple AI interactions** → **immediate response**
- **Heavy AI operations** → **Redis + BullMQ** → **Python Worker**

So:

```
Simple chat/question
→ Direct
→ Fast response

Heavy image/case/RAG processing
→ BullMQ
→ Python Worker
→ AI processing
```

Decision 22 — LOCKED

Conservative asynchronous architecture. We use the queue where it provides a real benefit, not for everything.

Decision 23 — Six fallback solutions

#	Problem	Fallback solution
1	Primary AI model fails	Retry once → switch to the common fallback model → if both fail, return a controlled AI-unavailable state and allow manual workflow
2	RAG fails	Retry retrieval → if still unavailable, do not fabricate evidence . For non-safety questions, the model may answer without RAG; for diagnosis/safety analysis, mark it as limited and send to human review
3	Backend tool fails	Retry once → if still failing, do not invent the result . Return the case to the human workflow. Example: if <code>rankTechnicians()</code> fails, dispatcher manually chooses the technician

#	Problem	Fallback solution
4	Python Worker / BullMQ fails	Automatically retry the job → if retries fail, mark job as FAILED , preserve the original request/case in PostgreSQL, notify the user/dispatcher, and allow manual processing
5	AI produces invalid structured output	Validate against schema → attempt one repair/regeneration → validate again → if still invalid, reject the AI output and use manual handling
6	Complete AI outage	Entire platform continues normally without AI. Customer can create a manual request, dispatcher can review/assign manually, and technician can complete the job normally

The important distinction

We shouldn't use the same fallback for everything.

Model failure → another model.

RAG failure → retry retrieval / limited response / human review.

Tool failure → retry / manual operation.

Worker failure → retry queued job / manual operation.

Bad AI output → validate / regenerate / reject.

Total AI failure → normal non-AI maintain workflow.

Decision 24 = LOCKED

maintain will use layered testing covering:

1. Backend/system functionality
2. Maintenance Agent
3. RAG retrieval/evidence
4. Operations Agent + technician ranking
5. Security + failure/fallback behavior

That completes **Decisions 16–24.**

Architecture/design decision phase: COMPLETE.